

六轴工业机器人嵌入式运动控制固件开发与仿真验证

第 1 周 系统方案设计与开发环境搭建

一、本周工作目标

本周主要目标是完成项目第 1 阶段的基础环境和总体框架搭建，为后续外设驱动、运动学、轨迹规划和 PID 控制开发建立可运行、可验证的基础平台。

二、本周工作完成情况

本周已基本完成第 1 阶段核心任务，建立由 Cortex-M4、FreeRTOS、QEMU、UART、Python、PyBullet 和 UR5 组成的嵌入式机器人开发与仿真验证平台。



三、本周主要工作内容

3.1 系统总体方案设计

首先对项目进行模块化拆分，确定系统由驱动 / BSP 层、通信层、算法层、应用层、Python 主机通信桥以及 PyBullet 机器人仿真端组成，并明确各层职责边界。Cortex-M4 / FreeRTOS 作为后续机器人控制算法的主要执行平台，Python 和 PyBullet 主要承担通信桥接、机器人本体仿真以及状态反馈功能。

本周同时完成《系统技术方案设计文档》，对软件架构、模块划分、通信链路和基础二进制协议进行了定义。

3.2 Cortex-M4 裸机开发环境搭建

安装并配置 ARM GNU Toolchain，使用 arm-none-eabi-gcc 建立 Cortex-M4 交叉编译环境，并选择 QEMU mps2-an386 作为 Cortex-M4 仿真平台。

为了使 Cortex-M4 程序能够独立启动，建立 startup.s、linker.ld 和 main.c 等基础文件。启动文件负责向量表、复位入口及 C 运行环境初始化，链接脚本负责 Flash、RAM 及程序数据段布局。

首先完成最小裸机程序验证，通过 UART 成功输出信息，确认交叉编译、ELF 加载、程序入口和 UART 基础输出均正常。

3.3 FreeRTOS 基础工程搭建

在裸机工程基础上加入 FreeRTOS Kernel，完成 FreeRTOSConfig.h、Cortex-M Port、SysTick / PendSV / SVC 异常入口、Heap 以及任务调度相关配置。

首先建立 Task A 和 Task B 两个基础任务，通过周期 UART 输出验证任务切换与 vTaskDelay()。随后将基础测试任务替换为 ProtocolTX 和 ProtocolRX，为双向通信建立实时任务运行基础。

```
ProtocolTX: 周期发送六轴目标角
ProtocolRX: 周期检查 UART 并解析状态帧

FreeRTOS Tick: 1 ms
ProtocolRX: 1 ms
ProtocolTX: 1000 ms
```

3.4 CMake 交叉编译系统建立

在基础工程能够通过手工编译命令运行后，将工程进一步整理为 CMake 构建系统，建立根目录 CMakeLists.txt、固件子目录 CMakeLists.txt 以及 cmake/arm-none-eabi-gcc.cmake 工具链文件。

```
cmake -S . -B build/cmake-arm -G Ninja --toolchain cmake/arm-none-eabi-gcc.cmake
cmake --build build/cmake-arm
```

CMake 能够统一管理启动文件、FreeRTOS Kernel、Cortex-M Port、协议模块以及 Board 支持代码，并生成供 QEMU 加载的 freertos_demo.elf。

3.5 Board Support Package 基础接口

为了避免硬件相关初始化代码长期集中在 main.c 中，本周建立 Board Support Package 基础接口，提供 board_clock_init() 和 board_gpio_init()。当前 QEMU mps2-an386 环境按 25 MHz 系统时钟配置，GPIO 部分完成工程接口与寄存器模板，为后续迁移实际 MCU 或进一步封装外设驱动预留统一入口。

3.6 PyBullet 与 UR5 仿真环境搭建

Python 端使用 PyBullet 建立六轴机器人仿真环境，完成 GUI 启动、地面加载、UR5 URDF、Visual / Collision Mesh、六个运动关节识别、单关节位置控制、环境物体添加和多视角切换。

```
shoulder_pan_joint
shoulder_lift_joint
elbow_joint
wrist_1_joint
```

```
wrist_2_joint
wrist_3_joint
```

仿真端使用 `p.setJointMotorControlArray(...)` 进行基础位置控制，并使用 `p.getJointState(...)` 获取六轴实际状态。

3.7 UART 二进制通信协议设计

为建立 Cortex-M4 与 Python 之间的结构化数据通信，设计固定格式二进制帧。

AA 55 | Command | Length | Payload | Checksum

关节数据采用 $6 \times \text{signed int16}$ ，单位为 0.01° ，单帧长度 17 Byte，多字节数据采用 Little Endian。当前定义 0x01、0x81 和 0x82 三个主要命令，分别用于目标角下发、实际状态上报和 MCU 接收确认。

Command	方向	功能
0x01	MCU → Python	六轴目标关节角
0x81	Python → MCU	六轴实际状态
0x82	MCU → Python	状态接收确认 ACK

3.8 双向通信与机器人联动

单向链路验证以 Joint 1 = 10.00° 为测试值。协议整数表示为 1000，即 0x03E8，Little Endian 字节序为 E8 03。Python 成功解析得到 [10.0, 0.0, 0.0, 0.0, 0.0, 0.0]，验证协议打包、字节序和 Checksum 正常。

在此基础上，将目标改为 Joint 1 = 60° ，其余关节为 0° 。Python 成功收到 0x01 目标帧并驱动 PyBullet 中的 UR5 第一轴运动。机器人运行过程中，Python 每约 0.2 s 读取六轴实际关节状态，并通过 0x81 JOINT_STATE 返回 Cortex-M4。

RX 0x01 目标角:

[60.0, 0.0, 0.0, 0.0, 0.0, 0.0]

TX 0x81 实际角度:

[60.0, -3.46, -0.02, -0.07, 0.69, -2.96]

RX 0x82 MCU 已确认状态:

[60.0, -3.46, -0.02, -0.07, 0.69, -2.96]

Cortex-M4 完成 Header、Command、Length 和 Checksum 检查后解析状态，并通过 0x82 返回 ACK，从而完成“控制指令下发 → UR5 执行 → 状态采集 → 状态反馈 → MCU 解析 → ACK 确认”的双向闭环。

四、本周主要问题及解决过程

4.1 FreeRTOS 无法直接在最小裸机工程中运行

初始裸机程序能够进入 main() 并完成 UART 输出，但加入 FreeRTOS 后还依赖完整的数据段初始化及异常处理。进一步完善 .data 初始化、.bss 清零、SVC、PendSV 和 SysTick 异常入口后，最终通过多个任务周期输出验证 FreeRTOS Scheduler 正常运行。

4.2 -nostdlib 环境缺少基础内存函数

固件采用 -nostdlib 链接，没有完整标准 C Runtime。FreeRTOS 相关代码需要 memset、memcpy，因此增加 memory.c 实现最小内存操作函数，解决链接依赖问题。

4.3 FreeRTOS Cortex-M Port 选择

项目目标为 Cortex-M4，但当前仿真不使用 FPU 上下文切换。最终采用 portable/GCC/ARM_CM3 Port 完成当前基础运行，并通过任务切换验证其可用性。

4.4 PyBullet 无法直接使用预编译包

当前 Python 环境未能直接获得匹配的 PyBullet Wheel，因此安装 Microsoft C++ Build Tools，在本地完成 PyBullet Native Extension 构建，随后 GUI 正常启动。

4.5 中文路径导致 PyBullet 资源加载异常

初始工程路径包含中文字符，PyBullet 底层 Native 模块在访问 plane.urdf 等资源时出现路径解析异常。将工程迁移至纯 ASCII 路径 D:\1_ToGo\Robo\code\six-axis-robot-motion-control 后恢复正常。

4.6 UR5 URDF 与 Mesh 资源不匹配

URDF Visual 部分引用 .dae 文件，但现有资源目录中的 Mesh 类型不匹配。最终使用 ROS-Industrial UR5 模型中的 visual/*.dae 和 collision/*.stl 补齐资源，UR5 Visual / Collision Model 均成功加载。

4.7 QEMU UART 与 Python 之间不存在直接接口

Cortex-M4 只能访问 QEMU 模拟 UART，Python 无法直接读取 MCU 内部 UART 对象。因此将 QEMU Serial Backend 映射为 tcp:127.0.0.1:5555，Python 使用 Socket 连接，形成 MCU UART → QEMU → TCP → Python 的桥接。

4.8 双向通信后的执行时序复杂化

加入状态反馈后，Python 需要同时处理 MCU 指令、PyBullet 仿真、状态采集和状态回传。Python 使用非阻塞 Socket 检查，避免网络接收阻塞约 240 Hz 的仿真循环；MCU 侧则拆分为 ProtocolTX / ProtocolRX 两个 FreeRTOS Task。

4.9 VS Code 未立即识别新安装的 CMake

外部 PowerShell 能够调用 cmake，但已打开的 VS Code 未继承更新后的 PATH。重新从环境变量已更新的终端启动 VS Code 后恢复正常。

4.10 CMake Toolchain 参数解析异常

使用 -DCMAKE_TOOLCHAIN_FILE=... 时当前环境出现路径参数解析异常，改用 --toolchain cmake/arm-none-eabi-gcc.cmake 后成功完成配置。

4.11 增加 BSP 后进行完整回归测试

在加入 CMake、Clock 初始化和 GPIO 初始化模板后，再次运行完整机器人控制链路。系统仍能同时观察到 RX 0x01、TX 0x81 和 RX 0x82，UR5 第一轴仍可到达约 60°，说明新增工程模块未破坏既有通信与仿真功能。

五、本周总结与收获

本周工作从 Cortex-M4 最小裸机程序开始，逐步建立 FreeRTOS 任务调度、UART 驱动、二进制协议、Python 通信桥以及 PyBullet / UR5 仿真环境。

开始阶段对上下位机系统的理解主要集中在“主机发送数据、仿真端接收数据”的简单交互模型。在实际搭建过程中进一步认识到，嵌入式机器人控制系统需要同时处理硬件抽象、实时任务调度、通信协议、数据同步、仿真执行和状态反馈等多个层次。

FreeRTOS Tick	1 ms
ProtocolRX	1 ms
ProtocolTX	1000 ms
PyBullet Simulation	≈240 Hz
State Feedback	200 ms

不同模块运行周期并不一致，因此系统不仅需要完成数据传输，还需要合理组织执行时序。通过本周工作，对 Cortex-M 启动流程、FreeRTOS 调度机制、嵌入式与主机通信以及机器人仿真系统之间的关系形成了更完整的认识。

六、结论

第 1 阶段计划内容已经基本完成。当前系统已经从最初的 Cortex-M4 最小裸机程序发展为能够运行 FreeRTOS、进行 UART 双向通信并驱动 PyBullet UR5 仿真机器人的基础控制平台。